Grefur Sensor

All rights reserved © Grefur

Hardware

Inputs

Outputs

Actuators bridge the gap between your logic and the physical world. Connect them together with a calculation and drive the result to a GPIO pin.

Control Modes

The device is fully configurable and can be set up for outputting both logical 1s and 0s, and fully analog signals.

Digital (High/Low)

Converts logical calculation values into a binary (On/Off) signal. The system interprets any value above **0.5** as active (High) and values equal to or below 0.5 as inactive (Low). This provides a robust transition from floating-point logic to physical switching.

- Best for: Controlling relays and digital inputs for fans, pumps and heating. Simple controls for light and solenoid valves.
- Key settings:
    - Invert logic:

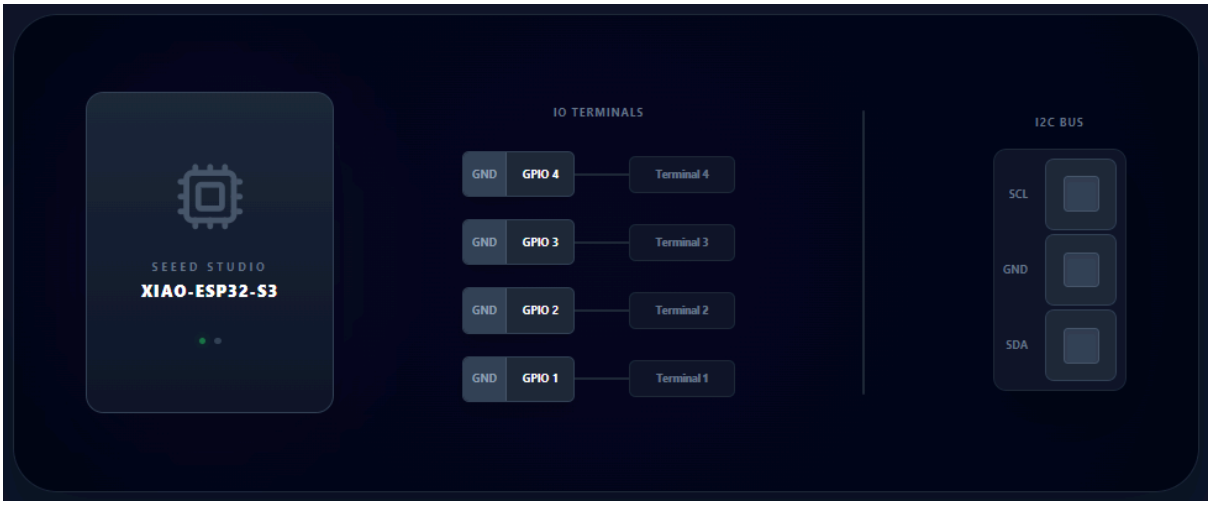
Analog (Voltage)

Generates a variable output signal. The system takes a logical value decided under engineering and maps it linearly to the hardware's 8-bit resolution (0–255) output. This emulates an analog voltage.

- Best for: Controlling speed of fans, pumps and motors. Dimming lights. Controlling valves and heaters using PID controllers.
- Key settings:
    - Min/Max Logical value
    - Min/Max Electrical value

All rights reserved © Grefur

Pinout diagram



General purpose pins are pulled high (5v) via a 47k ohms resistor.

All rights reserved © Grefur

Software

Expressions

The **Grefur Expression Engine** allows you to transform raw sensor data into intelligent actions. Whether you want to calculate the average temperature, create a thermostat, or build a complex controller, **Expressions** make it possible through a simple, text-based interface.

Calculation Modes

When setting up logic on your device, you can choose how the "brain" processes information

Event-Based (Expression)

These calculations only trigger when the input data actually moves.

- **Best for:** General monitoring, simple math (e.g., converting Celsius to Fahrenheit), or calculating values that don't change often.
- **Key Settings:**
  - **`Trigger Threshold`** – Defines how much the result must change before a new value is sent to your data lake.

Continuous (Fixed Interval)

These run strictly on a heartbeat timer (e.g., every 500ms).

- **Best for:** Automation and Control (like PID controllers) where timing must be precise to maintain stability.
- **Key Settings:**
  - **`Evaluation Interval`** – How often (in milliseconds) the calculation should run.

External (Remote / MQTT binding)

These values are collected from an external source. Event based renewal.

- **Best for:** Collecting measurements from other sensors
- **Key Setting:**
  - **Binding Tag (MQTT Topic or ID)** ***Case Sensistive*** – Full topic for the requested value
  - **Reset Interval (ms)** – Time of one new update from external source
  - **Fallback Value** – Fallback value if reset interval is triggered

All rights reserved © Grefur

Addon Properties

Every named expression exposes internal state as readable properties using dot notation. This allows you to build logic that reacts not just to *what* a function outputs, but to *how* it is behaving internally without adding extra sensors or calculations.

The syntax is **`ExpressionName.property`**, and it can be used anywhere a value can be used: in other expressions, as actuator inputs, or combined with logic operators.

Generic values on every named expression regardless of type.

Property	Value to return
`.name`	The name of the expression of a string.
`.value`	Last evaluation output value.
`.lastRun`	Milliseconds since the expression was last evaluated

Different functions expose different properties. The available properties for each function are listed together with the function's own documentation below.

All rights reserved © Grefur

Smart Functions

Powerful built-in functions to handle automation tasks decentralized to ensure autonomous control even if the cloud drops out.

PID Controller

Used for precise control, such as keeping a room at an exact temperature by adjusting a fan or heater.

- **Format:** `PID(input,setpoint,Kp,Ki,Kd)`
- **Example:** `PID(RoomTemp,22.5,10,0.5,2)`
- **What it does:** It looks at the current temperature (`RoomTemp`) and the goal (`22.5`) and calculates how much power to apply based on the Tuning Constants (Kp, Ki, Kd).

Properties

- `.error` Current error (setpoint − input)
- `.integral` Accumulated integral sum

Hysteresis (Stability Guard)

Prevents actuators from "chattering" or "flapping" by using two different thresholds for turning ON and OFF.

- **Format:** `HYST(input,highTreshold,lowTreshold)`
- **Example:** `HYST(Temp01,18,24)`
- **What it does:** If temperature is measured below 18 degrees, it will activate the output. When the temperature is above 24 degrees, it will deactivate.

CYCLE (Asymmetrical Pulse)

Counts rising edges on digital inputs. Can count up, down, and reset on demand.

- **Format:**
  - `COUNT(Up)`
  - `COUNT(Up,Down)`
  - `COUNT(Up,Down,Reset)`
- **Example:** `COUNT(FlowPulse,0, DayShift)`
- **What it does:** Increments by one for each pulse indicating flow. Reset midnight.

All rights reserved © Grefur

DELAY (Timers)

Ensures a value is stable before passing it through. Perfect for ignoring brief sensor spikes or adding intentional delays to control logic.

- **Format: `DELAY(input,time)`**
- **Format: `DELAY(input,time,unit)`**
- **Format: `DELAY(input,time,method)`**
- **Format: `DELAY(input,time,method,unit)`**

Arguments:

- input — Any value or expression to monitor
- time — Duration as a number
- mode — ON, OFF, or omitted (see modes below)
- unit — ms, s, min, h (default: ms)

Units (case-insensitive):

ms / milliseconds — e.g. DELAY(Input, 500, ms)

s / sec / seconds — e.g. DELAY(Input, 5, s)

m / min / minutes — e.g. DELAY(Input, 2, min)

h / hour / hours — e.g. DELAY(Input, 1, h)

Modes:

No mode — Stable delay Waits for input to remain unchanged for the full duration before passing it through. Any change resets the timer. Example: DELAY(WaterLevel > 80, 5, s) Returns 1 only if WaterLevel has been above 80 continuously for 5 seconds. If it drops for even a millisecond, the timer resets.

ON — Delay-on timer Delays the ON signal. Output goes high only after input has been 1 for the full duration. Turns OFF immediately when input goes low. Example: DELAY(PumpRequest, 10, s, ON) Output turns on only after PumpRequest has been 1 for 10 continuous seconds. If input drops, output turns off immediately and the timer resets.

OFF — Delay-off timer Turns ON immediately when input goes high. Keeps output high for the set duration after input drops low. Example: DELAY(FanRequest, 30, s, OFF) Output turns on instantly. When FanRequest drops to 0, output stays on for 30 more seconds before turning off.

Example

- **`DELAY(WaterLevel>80,5000,ms)`**
- **What it does:** Returns only, only if the measured waterlevel is above 80 for 5 continuous seconds. If the level drops for even a millisecond, the timer resets.

All rights reserved © Grefur

Properties

- `.timeLeft` Milliseconds remaining until the output changes
- `.active` 1 if the timer is currently counting, 0 if idle

Set-Reset Latch(One bit of memory)

Stores a state for multiple iterations and makes it possible to use other criteria to reset it.

- **Format:** `SR(signalOn,signalOf)`
- **Example:** `SR(userButton1,lightlevel<1000lux)`
- **What it does:** Runs a signal when the user presses a button. Kee it on until the level of light goes above 1000lux level.

COUNT (Pulse counting)

Generates a repeating ON/OFF pattern. Ideal for dosing pumps or blinkers.

- **Format:** `CYCLE(onTimeMs,offTimeMs) / CYCLE(timeMs)`
- **Example:** `CYCLE(2000,8000) / CYCLE(5000)`
- **What it does:** Runs a signal on for 2000 ms and off for 8000 ms forever. If only one value is provided, it will create a 50/50 pulse.

Properties

- `.up` Raw pin state of the up input (0 or 1)
- `.down` Raw pin state of the down input (0 or 1)
- `.reset` Raw pin state of the reset input (0 or 1)

RATE (Frequency measurement)

Measures how many rising edges occur per second over a sliding time window.

- **Format:** `RATE(digitaInput, timePeriodeMs)`
- **Example:** `RATE(FlowPulse,10000)`
- **What it does:** Counts rising edges within the given time window and returns edges per second. Larger windows give smoother readings but respond slower to changes.

Properties

- `.edgeCount` Number of pulses counted in the current window
- `.windowAge` Milliseconds elapsed since the current window started

All rights reserved © Grefur

INTEGRATE (Accumulator)

Accumulates a value multiplied by elapsed time. Resets when the reset input goes high.

- **Format:**
  - `INTEGRATE(input)`
  - `INTEGRATE(input, reset)`
- **Example:** `INTEGRATE(HeatPower,DayShift)`
- **What it does:** Multiplies the input by the time elapsed since the last evaluation (in hours) and adds it to an internal sum. If input is in kW, the result is kWh. Reset can be a button, a `CYCLE()` timer, or any expression.

Properties

- `.total` Current accumulated value
- `.runTime` Milliseconds since the last reset

AVERAGE (Smoothing)

Calculates the average of multiple sensors or values.

- **Format:** `AVG(n1,n2,n3,…)`
- **Example:** `AVG(Temp01,Temp02,Temp03,...)`
- **What it does:** Smooths out data by averaging multiple inputs, giving you a more reliable "Room Temperature" reading.

ROUND (Data Cleaner)

Cleans up messy sensor data for displays.

- **Format:** `ROUND(value,decimals)`
- **Example:** `ROUND(Temp01,2)`
- **What it does:** Makes a number with decimal to a locked two decimal number.

LIMIT (Safety Guard)

Prevents values from exceeding safe operational boundaries.

- **Format:** `LIMIT(value,min,max)`
- **Example:** `LIMIT(HeaterPower,0,100)`
- **What it does:** If the power calculation tries to go to 110%, it forces it back to 100%.

All rights reserved © Grefur

CURVE (Smart Mapping)

Perfect for non-linear data, like measuring the volume of a round fuel tank or creating custom light-dimming profiles.

- **Format:** `CURVE(input,x1,y1,x2,y2, ...,applyLimit)`
- **What it does:** You provide pairs of points (Input, Output). Intelligence draws a line between them to find the correct result.

RAMP (Intelligent adaption)

Prevent sudden jumps in data output. The function ensures that values transition smoothly over time, making it ideal for motor soft-starts, dimming profiles, or filtering "noisy" sensor data.

- **Format:** `RAMP(target,rate)`
- **What it does:** Increasing and decreasing the target value at given rate (maximum change per second)

- **Format:** `RAMP(target,rateUp,rateDown)`
- **What it does:** Uses individual speeds (maximum change per second) for both increasing and decreasing the target value

Properties

- `.current` Current ramp output value
- `.delta` Remaining distance to target

Logic & Math

- **Ternary (IF/THEN):** `Condition ? ValueIfTrue : ValueIfFalse`
  - *Example:* `Temp>30?1:0` (Turn a switch ON if it's too hot).
- **Math:** `ABS()` (Absolute value), `SQRT()` (Square root), and basic arithmetic (`+`, `-`, `*`, `/`).

Static values

- To create one, you simply **type the number directly into the expression field**. The engine recognizes this as a constant, unchanging output. The value can be published when it changes and be used in other logic.
- Ex: `20`

All rights reserved © Grefur

Reference Data

Link data across your network without memorizing complex hardware IDs is fully possible.

- **Local sensors:** Use the name you gave your sensor.

  Ex: **`OutdoorTemp`**, just type **`OutdoorTemp*1.1`** in your expression.

- **Numeric Values:** Use numbers as offsets or fixed limits.

  Ex: **`OutdoorTempOffset`**, just type **`OutdoorTempOffset*1.1`** in your expression.

- **External (MQTT Binding)**: Objects of type “Binding” can be bound to expressions using the name of the binding.
  - Ex: **`RunExternalFan`**, just type **`RunExternalFan?1:0`** in your expression.

Safety and Validation

To prevent errors in your automation, the Grefur engine automatically checks your work:

- **Parentheses Check:** Ensures every **`(`** has a matching **`)`**.
- **Logic Check:** Ensures your "If/Then" statements are complete.
- **Safety Clamps:** In Continuous mode, you can set a **`Min Value`** and **`Max Value`** that the engine will never exceed, regardless of what the math says.

All rights reserved © Grefur